DYNASOLVER: A NEURAL OPERATOR FOR AUTOREGRESSIVE 4D FLOW PREDICTION WITH DYNAMIC BOUNDARIES

Rui Qian[1,*] **Shibo Dai**[1,*] **Mengjun Lin**[1,*] **Yuk-Kit Hou**[2]
Saining Zhang[3] **Junhao Chen**[1] **Hao Zhao**[1] **Chaojian Li**[4,†]

[1]Tsinghua University [2]Peking University [3]Nanyang Technological University
[4]The Hong Kong University of Science and Technology

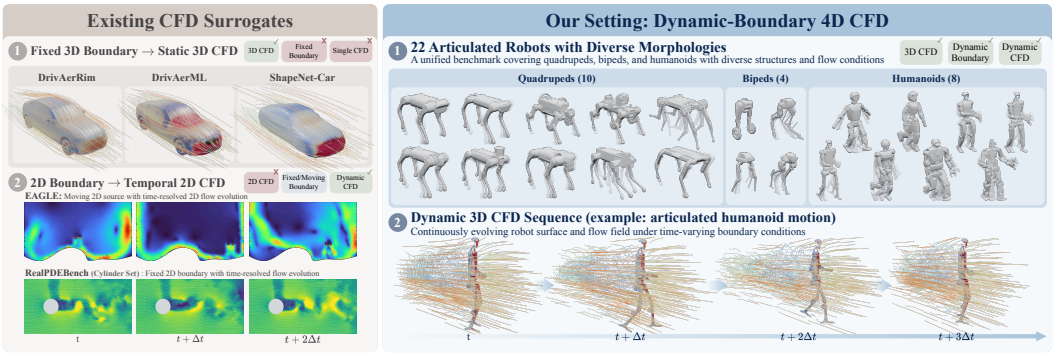


Figure 1: **Existing CFD Settings versus Dynamic-Boundary 4D CFD.** Existing CFD surrogates typically model either static three-dimensional geometries or temporally evolving two-dimensional flows. In contrast, RoboCFD-4D requires joint modeling of articulated three-dimensional boundary motion and the resulting time-evolving flow field across diverse robot morphologies.

ABSTRACT

Neural operators provide efficient surrogates for computational fluid dynamics (CFD), but most are designed for static geometries and are not equipped to model unsteady flows induced by continuously moving boundaries. We study *dynamic-boundary 4D CFD prediction* for articulated moving geometries and introduce **RoboCFD-4D**, a temporally resolved robotic CFD benchmark spanning diverse morphologies and flow conditions. To address the computational and autoregressive challenges of this setting, we propose **DynaSolver**, a neural operator that explicitly conditions on boundary evolution and causal flow history. DynaSolver employs an efficient physical-token formulation that amortizes point-to-token projection across the operator stack, substantially reducing repeated point-space spatio-temporal computation while retaining accurate point-level reconstruction. We further introduce Replay-Augmented Parallel Training, which exposes the model to autoregressively generated histories while preserving parallel dense supervision, narrowing the gap between ground-truth-conditioned training and autoregressive rollout. Experiments on RoboCFD-4D show that DynaSolver achieves the best full-rollout accuracy among a broad range of learned CFD surrogates, with its advantage increasing as rollout proceeds, indicating substantially reduced autoregressive error accumulation.

1 INTRODUCTION

Computational fluid dynamics (CFD) is a fundamental tool for analyzing complex flow phenomena, and recent learning-based surrogates, particularly neural operators, have substantially reduced the computational cost of high-fidelity flow prediction. Most existing progress (Li et al., 2021; Liu

[*]Equal contribution. [†]Corresponding author.

Table 1: Comparison of representative CFD learning datasets across spatial dimensionality, temporal resolution, and boundary dynamics. "Dim." denotes spatial and temporal dimensionality, where 3D+t indicates temporally resolved three-dimensional CFD.

Dataset	Domain	Dim.	Boundary Dynamics	Sequences	CFD States
AirfRANS (Bonnet et al., 2022)	Airfoil aerodynamics	2D	Static solid	N/A	1,000
CFDBench (Luo et al., 2023)	General fluid dynamics	2D+t	Static solid	739	301,671
MegaFlow2D (Xu et al., 2023)	External / internal flow	2D+t	Static solid	$\sim$3,000	$>$2M
RealPDEBench (Hu et al., 2026a)	real scenarios	2D+t	Static/Moving solid	736	$\sim$2.6M
Eagle (Janny et al., 2023)	Drone	2D+t	Moving Source, Static solid	1184	$\sim$1M
ShapeNet-Car (Umetani & Bickel, 2018)	Automotive aerodynamics	3D	Static solid	N/A	889
NASA CRM (Bekemeyer et al., 2025)	Aircraft aerodynamics	3D	Static solid	N/A	121
AhmedML (Ashton et al., 2024a)	Bluff-body aerodynamics	3D	Static solid	N/A	500
DrivAerML (Ashton et al., 2024b)	Automotive aerodynamics	3D	Static solid	N/A	500
DrivAerRim (Xia et al., 2026)	Automotive aerodynamics	3D	Static solid	N/A	904
DrivAerNet (Elrefaie et al., 2025)	Automotive aerodynamics	3D	Static solid	N/A	4,000
DrivAerNet++ (Elrefaie et al., 2024)	Automotive aerodynamics	3D	Static solid	N/A	$\sim$8,000
RoboCFD-4D (Ours)	**Robot aerodynamics**	**3D+t**	**Articulated moving solid**	**352**	**10560**

et al., 2026; Curvo et al., 2026; Hu et al., 2026b; Alkin et al., 2025; Wu et al., 2024), however, has focused on static aerodynamic configurations such as airfoils (Bonnet et al., 2022), vehicles (Elrefaie et al., 2025; 2024; Alkin et al., 2025), and fixed industrial components (Li et al., 2023b; Deng et al., 2024), where the geometry remains unchanged during prediction. Meanwhile, robotic systems are increasingly operating in high-speed, outdoor, and fluid-interactive environments (Paolino et al., 2025; Loquercio et al., 2021), where aerodynamic effects can directly influence locomotion stability and control (Jones et al., 2022; Lale et al., 2024). Efficient prediction of the surrounding flow field is therefore becoming increasingly relevant to robotic systems (Du et al., 2021; Hu et al., 2019). Unlike conventional aerodynamic objects, however, robots are articulated systems whose geometry evolves continuously during motion, fundamentally violating the static-boundary assumption underlying most existing CFD surrogates (Wu et al., 2024; Li et al., 2023b; Cao et al., 2023; de Avila Belbute-Peres et al., 2020).

Flow prediction around a moving robot cannot be reduced to independently applying a static CFD surrogate to each instantaneous geometry. Under prescribed motion, the evolving boundary continuously drives the surrounding fluid (Peskin, 2002; Sotiropoulos & Yang, 2014), while previously developed flow structures persist over time. More generally, finite or partially observed dynamical representations can exhibit memory effects (Parish & Duraisamy, 2017; Ayed et al., 2019), so similar robot configurations may correspond to different flow states depending on their preceding evolution. We therefore formulate *dynamic-boundary 4D CFD prediction*, where both the three-dimensional boundary and surrounding flow evolve over time.

Learning such dynamics introduces three challenges beyond conventional static-boundary CFD prediction. First, the model must represent both boundary evolution and causal flow context rather than treating each geometry independently. Second, extending neural operators to temporally resolved three-dimensional CFD substantially increases computational cost, since repeated point-space processing compounds across spatial resolution, temporal context, and network depth. Third, autoregressive rollout recursively feeds predicted states back into the model, creating a mismatch with ground-truth-conditioned training and causing prediction errors to accumulate over time (List et al., 2025; Brandstetter et al., 2022; Kochkov et al., 2021). Dynamic-boundary CFD therefore requires a model that captures causal boundary-flow evolution, performs spatio-temporal reasoning efficiently, and remains robust under closed-loop rollout.

To enable systematic study of this problem, we construct **RoboCFD-4D**, a temporally resolved three-dimensional CFD benchmark based on articulated robotic motion. As summarized in Table 1, existing CFD learning datasets predominantly consider static three-dimensional geometries (Elrefaie et al., 2025; 2024; Bonnet et al., 2022; Alkin et al., 2025) or temporally evolving two-dimensional flows (Janny et al., 2023; Hu et al., 2026a), whereas RoboCFD-4D contains 352 sequences and 10,560 CFD frames with articulated moving three-dimensional boundaries. Building on this benchmark, we propose **DynaSolver** based on a simple principle: dynamic-boundary CFD should be modeled as causal evolution between physical flow states under prescribed boundary motion, rather than as independent geometry-to-flow predictions. DynaSolver therefore combines explicit boundary evolution and causal flow history with an efficient physical-token temporal operator, enabling compact spatio-temporal reasoning without repeatedly returning to the full point representation. We further introduce *Replay-Augmented Parallel Training*, which augments clean training histories with

autoregressively generated states while retaining parallel dense supervision, improving rollout robustness without the cost of sequential student optimization.

Our main contributions are summarized as follows:

- We formulate dynamic-boundary 4D CFD prediction for temporally evolving three-dimensional flow fields and introduce **RoboCFD-4D**, containing 352 sequences and 10,560 time-resolved CFD frames with articulated moving boundaries.
- We propose **DynaSolver**, a neural operator that jointly models explicit boundary evolution and causal flow dynamics. Its physical-token temporal operator eliminates layer-wise point–token alternation, performing repeated spatio-temporal reasoning in compact token space with only a single slice/deslice pair.
- We introduce **Replay-Augmented Parallel Training**, which combines autoregressively generated histories with parallel dense supervision, exposing the model to rollout states without the sequential student optimization required by dense autoregressive training.

2 Related Work

CFD datasets with evolving dynamics. Existing CFD learning benchmarks span a wide range of geometries but remain largely dominated by static or simplified dynamic settings. AirfRANS provides steady 2D RANS solutions over airfoils, while DrivAerNet and DrivAerNet++ scale high-fidelity CFD to large collections of static 3D vehicle geometries (Bonnet et al., 2022; Elrefaie et al., 2025; 2024). Paolino (Paolino et al., 2025) extend aerodynamic prediction to articulated humanoid configurations, but adopt a quasi-steady assumption rather than resolving transient flow evolution. Time-resolved benchmarks such as EAGLE (Janny et al., 2023) study 2D flows driven by a moving source, while RealPDEBench includes several 2D moving-body scenarios (Hu et al., 2026a); unsteady pitching-airfoil simulations further consider prescribed motion with comparatively simple geometry (Baldan & Guardone, 2024). In contrast, RoboCFD-4D provides transient 3D CFD trajectories around complex articulated bodies with continuously evolving boundaries.

Learning-based CFD surrogates. Neural operators provide an efficient framework for learning mappings between physical fields. DeepONet (Lu et al., 2021) uses branch trunk factorization, while Fourier-based approaches such as FNO (Li et al., 2021) and its variants model global interactions through spectral representations (Lu et al., 2021; Li et al., 2021; Rahman et al., 2023; Wen et al., 2022; Hu et al., 2025; Tran et al., 2023; Wu et al., 2023). Geometry-aware operators, including GNO, Geo-FNO, and GINO, extend operator learning to irregular domains (Li et al., 2020; 2023a;b), while attention-based methods such as GNOT and Transolver provide direct global interactions on irregular discretizations (Hao et al., 2023; Wu et al., 2024). Mesh-based simulators such as GNS, MeshGraphNet, and BSMS-GNN instead propagate local interactions through graph message passing (Sanchez-Gonzalez et al., 2020; Pfaff et al., 2021; Cao et al., 2023). Physics-informed approaches further incorporate governing equations into training objectives (Cuomo et al., 2022; Li et al., 2024). Building on these advances, DynaSolver focuses on efficient spatio-temporal operator learning under continuously evolving articulated boundaries.

Autoregressive training for neural simulation. Autoregressive neural simulators are prone to error accumulation because model predictions are recursively reused as future inputs. Noise injection approximates this rollout corruption by perturbing ground-truth states during training (Sanchez-Gonzalez et al., 2020), while unrolled training exposes the model directly to its own predictions at the cost of sequential rollout and increased optimization expense. The pushforward strategy reduces gradient cost by detaching intermediate predictions while retaining autoregressive state generation (Brandstetter et al., 2022). Subsequent work further distinguishes the effects of rollout-distribution exposure and long-range gradient propagation by comparing one-step, detached, and fully differentiable unrolled training (List et al., 2025). Our Replay-Augmented Parallel Training instead decouples autoregressive history generation from student optimization, combining model-generated replay states with parallel dense supervision.

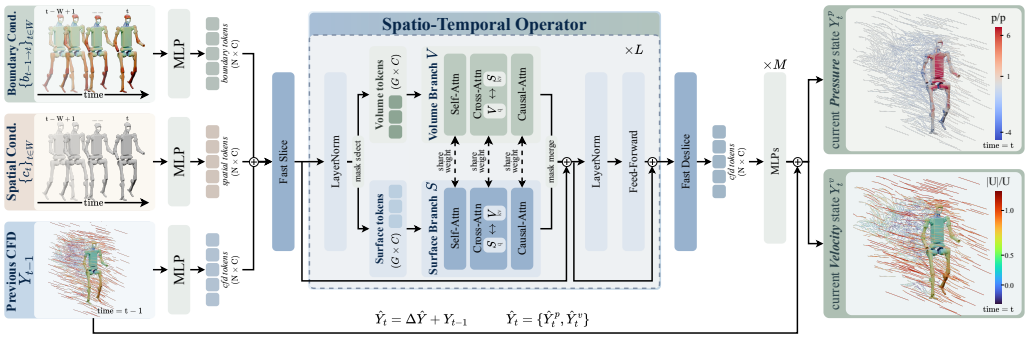


Figure 2: **Overview of DynaSolver.** Given the spatial condition, boundary evolution, and previous CFD states over a causal window, DynaSolver lifts and fuses the point-wise inputs before mapping them into separate surface and volume physical tokens. A single Fast Slice/Deslice pair encloses the spatio-temporal operator stack, where self-attention, cross-attention, and causal attention jointly model spatial and temporal flow interactions. The reconstructed point features are decoded into a residual flow field and added to the previous CFD state to predict the current velocity and pressure.

3 METHOD

We study dynamic-boundary 4D CFD prediction for articulated robots under prescribed motion, predicting the evolution of the surrounding flow from the current state and known boundary motion. After formalizing the problem in Sec. 3.1 and introducing the pipeline in Sec. 3.2, we present the token-space temporal operator in Sec. 3.3 and Replay-Augmented Parallel Training in Sec. 3.4.

3.1 PROBLEM SETUP

We consider one-way coupled transient incompressible flow around an articulated robot undergoing prescribed motion, where the moving robot boundary drives the surrounding fluid dynamics while the robot motion itself is not affected by the predicted flow. The surrogate operates on a fixed Eulerian model grid $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^{N}$. A set of tracked material points follows the robot surface and is projected onto $\mathcal{X}$ to define time-dependent surface-associated masks and geometric and kinematic features. Grid nodes occupied by the moving robot are excluded through a time-dependent validity mask M_t^v. The CFD state is represented on the fixed grid as $Y_t = [u, v, w, p]_t \in \mathbb{R}^{N\times 4}$, where (u, v, w) and p denote velocity and kinematic pressure, respectively.

Under a prescribed moving boundary, advancing the flow from $t-1$ to t requires not only the previous CFD state but also the time-dependent boundary configuration and kinematics (Peskin, 2002; Sotiropoulos & Yang, 2014). We therefore condition the prediction on a spatial condition $c_t \in \mathbb{R}^{N\times 11}$ describing the configuration at time t, and a boundary-evolution feature $b_{t-1\to t} \in \mathbb{R}^{N\times 8}$ describing the boundary motion over the transition, yielding the one-step mapping $(c_t, b_{t-1\to t}, Y_{t-1}) \mapsto Y_t$.

Although the complete CFD state under prescribed boundary evolution admits a local Markov transition, projection onto a finite or partially observed representation can introduce memory effects (Parish & Duraisamy, 2017; Ayed et al., 2019). Our surrogate operates precisely in such a setting, with time-varying boundary features and validity masks defined on a fixed Eulerian representation, and a single observed state is therefore not guaranteed to be Markov-sufficient. We consequently augment the transition with a short causal history. Defining the transition-aligned input $q_i = (c_i, b_{i-1\to i}, Y_{i-1})$, we collect at most W consecutive transitions as

$$\mathcal{Q}_t = \{q_i\}_{i=\max(1,t-W+1)}^{t}. \tag{1}$$

We then learn a temporal operator $\mathcal{F}_\theta$ over this causal context and predict the flow evolution in residual form,

$$\widehat{Y}_t = Y_{t-1} + \mathcal{F}_\theta(\mathcal{Q}_t). \tag{2}$$

Starting from the given initial state Y_0, the causal context grows until W transitions are available and then proceeds with a fixed-length sliding window.

3.2 METHOD OVERVIEW

Dynamic 4D CFD introduces two challenges beyond spatial flow prediction. First, temporal reasoning over multiple flow states substantially amplifies the cost of point-space computation. Second, efficient parallel training relies on ground-truth histories, whereas autoregressive inference recursively conditions on model predictions, creating a train–inference distribution gap. DynaSolver addresses these challenges with an efficient token-space temporal operator and Replay-Augmented Parallel Training, respectively.

Given the causal transition window $\mathcal{Q}_t$, the three input modalities of each transition are first lifted into a shared latent space,

$$H_i = \phi_c(c_i) + \phi_b(b_{i-1\to i}) + \phi_y(Y_{i-1}), \tag{3}$$

where ϕ_c, ϕ_b, and ϕ_y are two-layer MLPs. As illustrated in Fig. 2, DynaSolver slices the fused point features once into separate surface and volume physical tokens, processes them with a stack of spatio-temporal operator blocks, and deslices them only before final point reconstruction. A lightweight MLP decodes the reconstructed point features into the residual field ΔY_t, which is added to Y_{t-1} to obtain $\widehat{Y}_t$. During training, a frozen replay model generates autoregressive histories, while the current model retains parallel, densely supervised optimization through Replay-Augmented Parallel Training.

3.3 EFFICIENT TOKEN-SPACE TEMPORAL OPERATOR

The original Transolver (Wu et al., 2024) alternates between point and physical-token representations within each operator layer. Since the slicing weights are recomputed from the updated point features, the physical-token representation can evolve as the features are progressively refined. Carrying this layer-wise point–token alternation into temporal CFD, however, becomes costly because each transition contains thousands of surface and volume queries. We therefore amortize the physical tokenization across the operator stack: point-wise features are sliced once into a compact physical-token representation, all spatio-temporal reasoning is performed in this space, and the resulting features are desliced only at the output, as illustrated in Fig. 2.

Token-space operator. We use the fast slicing operation of Transolver3 (Zhou et al., 2026) to compress the lifted point features into separate surface and volume physical-token streams,

$$Z_i^{\mathrm{s}}, Z_i^{\mathrm{v}} = \mathrm{FastSlice}(H_i), \qquad Z_i^{\mathrm{s}}, Z_i^{\mathrm{v}} \in \mathbb{R}^{G\times D}. \tag{4}$$

The resulting tokens are processed by L spatio-temporal operator blocks. Each block applies intra-domain self-attention, bidirectional cross-attention between the surface and volume branches, causal temporal attention, and a token-space feed-forward network. Causal temporal attention (Vaswani et al., 2017) uses a block-causal mask over transitions, allowing each transition to aggregate information only from the current and preceding transitions.

Lightweight point reconstruction. After the final operator block, the surface and volume streams are mapped back to their corresponding query points through a single fast deslicing operation,

$$\widetilde{H}_i = \mathrm{FastDeslice}\left(Z_i^{\mathrm{s}}, Z_i^{\mathrm{v}}\right). \tag{5}$$

Because this restructuring removes the repeated point-wise refinement of the original backbone, a linear prediction head is insufficient to recover the point-level flow field. We therefore use a lightweight MLP decoder ϕ_{dec},

$$\Delta Y_i = \phi_{\mathrm{dec}}(\widetilde{H}_i), \tag{6}$$

and add the residual to the previous flow state following Eq. 2. The lightweight decoder compensates for the reduced point-level expressiveness introduced by the token-space restructuring, allowing the computational benefits of compact token processing to be realized without sacrificing prediction accuracy. As shown in Sec. 4.3, it recovers the performance degradation of the token-only variant with only modest additional overhead.

3.4 REPLAY-AUGMENTED PARALLEL TRAINING

Autoregressive flow prediction requires balancing rollout robustness with training efficiency. As illustrated in Fig. 3, *Dense Unrolled Training* (List et al., 2025) exposes the model to its own predictions by sequentially unrolling the trajectory, but repeatedly evaluates highly overlapping temporal

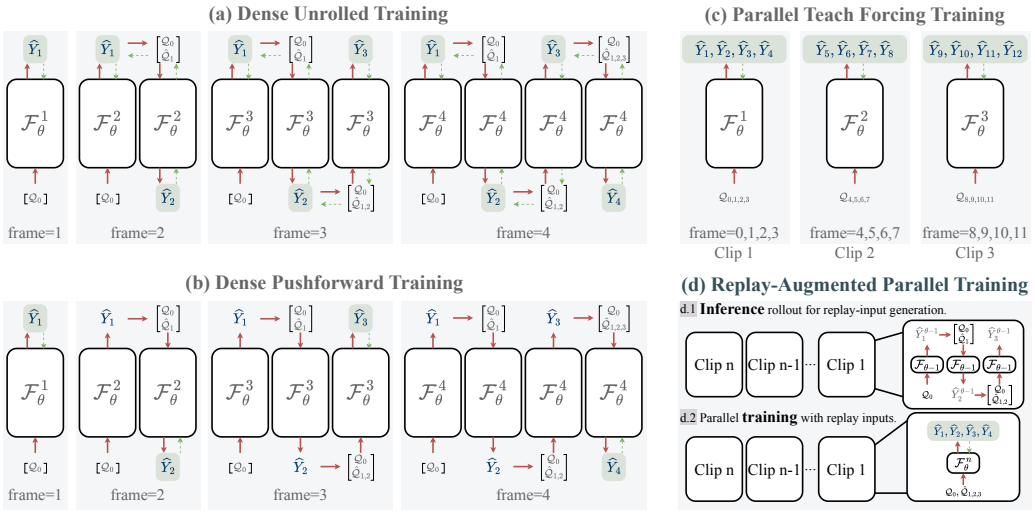


Figure 3: **Comparison of training strategies for autoregressive flow prediction.** Dense Unrolled Training (a) and Dense Pushforward Training (b) construct autoregressive histories sequentially, whereas Parallel Teacher-Forcing Training (c) predicts all transitions in a clip simultaneously from ground-truth histories. Our Replay-Augmented Parallel Training (d) decouples autoregressive history generation from student optimization: a frozen model first rolls out replay inputs (d.1), which are then used for parallel dense supervision of the current model (d.2). Red solid arrows → denote feedforward computation, while green dashed arrows ⇢ denote backpropagation.

contexts and incurs substantial training cost. *Dense Pushforward Training* (Brandstetter et al., 2022) reduces the backward cost by detaching preceding rollout steps, yet still requires sequential rollout construction. In contrast, *Parallel Training* (Vaswani et al., 2017) exploits block-causal attention to predict and supervise all transitions in a clip with a single forward pass, but conditions only on ground-truth histories and therefore suffers from a train–inference distribution gap. To combine autoregressive exposure with efficient parallel optimization, we introduce *Replay-Augmented Parallel Training*: a frozen model first performs inference rollout to generate replay inputs (Fig. 3d.1), after which the current model is optimized in parallel using the replay-augmented clips (Fig. 3d.2).

Replay generation. We partition each trajectory into non-overlapping transition clips of length W. At epoch $e > 1$, the previous-epoch model is frozen as the replay model, $\bar{\theta}^{(e)} = \theta^{(e-1)}$. For a fraction ρ of the clips, the frozen model starts from the ground-truth initial state and rolls out autoregressively using the same causal-window mechanism as inference:

$$\widetilde{Y}_i = widetildeY_{i-1} + \mathcal{F}_{\bar{\theta}^{(e)}}\left(\widetilde{\mathcal{Q}}_i\right), \tag{7}$$

where $\widetilde{\mathcal{Q}}_i$ follows the same growing-prefix and sliding-window construction as $\mathcal{Q}_i$, with ground-truth flow histories replaced by recursively generated replay states. Replay generation is detached from gradient computation, and the replay model remains fixed within each epoch.

Parallel training and objective. Unlike dense autoregressive training, the generated rollout is used only to construct input histories. As highlighted in Fig. 3, replay clips and the remaining clean ground-truth clips are then processed identically by the current model: all transition positions are predicted and supervised simultaneously in a single block-causal forward pass, while the targets remain ground-truth CFD states. For the supervised transition set $\mathcal{T}$, we optimize

$$\mathcal{L} = \frac{1}{|\mathcal{T}|}\sum_{i\in\mathcal{T}} \frac{\sum_{n=1}^{N} m_{i,n}\left\|\widehat{Y}_{i,n} - Y_{i,n}\right\|_2^2}{\sum_{n=1}^{N} m_{i,n}}, \tag{8}$$

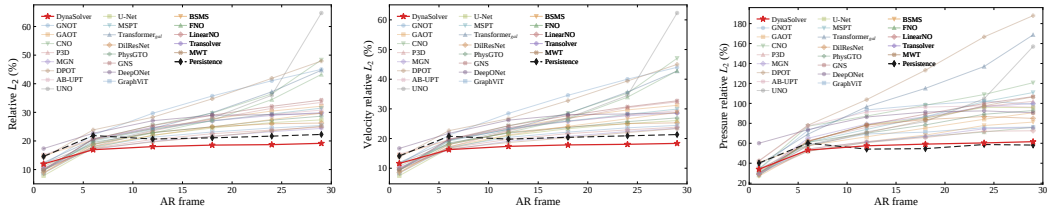
where $m_{i,n}$ denotes the validity mask. Mixing replay and clean histories exposes the model to realistic model-induced errors while retaining a clean-state anchor, and the frozen previous-epoch model keeps the replay distribution fixed within each epoch.

Table 2: **Quantitative comparison across rollout horizons on RoboCFD-4D Base.** We report the relative L_2 errors (%) of velocity and pressure at selected rollout steps, together with their full-rollout averages and the overall error. The best results among learned methods are highlighted in **bold**, while the second-best are underlined. Persistence is excluded from ranking.

Method	Per-frame Velocity ↓						Per-frame Pressure ↓						Avg. Vel. ↓	Avg. Pre. ↓	Overall ↓
	1	6	12	18	24	29	1	6	12	18	24	29			
GNOT (Hao et al., 2023)	11.38	21.14	28.47	34.58	40.01	43.99	34.08	72.89	93.49	98.66	102.21	99.18	30.56	87.84	31.55
GAOT (Wen et al., 2025)	14.46	19.54	23.58	26.93	29.66	31.19	40.96	57.70	66.52	75.18	84.08	90.67	24.60	70.25	25.32
CNO (Raonic et al., 2023)	9.32	19.36	24.27	28.49	35.25	47.04	34.24	77.09	87.29	98.14	108.89	120.71	27.12	91.52	28.28
P3D (Holzschuh et al., 2026)	9.32	17.07	22.62	26.70	30.44	32.36	32.55	62.66	76.03	85.25	96.86	102.08	23.72	78.82	24.61
MGN (Pfaff et al., 2021)	9.49	19.70	23.69	25.88	27.66	28.55	29.32	65.02	79.25	86.55	96.78	99.86	23.48	79.44	24.51
DPOT (Hao et al., 2024)	13.66	22.65	26.70	32.77	39.39	44.98	42.45	78.12	103.77	133.25	166.67	187.97	30.30	121.10	32.08
AB-UPT (Alkin et al., 2025)	14.75	19.60	23.36	25.68	27.92	29.38	42.29	77.47	91.30	96.30	97.54	96.23	23.83	86.84	25.03
UNO (Rahman et al., 2023)	11.71	18.17	22.82	26.91	34.53	62.31	34.13	58.36	71.43	81.76	104.69	156.99	27.32	81.55	28.40
U-Net (Ronneberger et al., 2015)	**7.48**	16.73	20.99	23.30	25.05	26.08	27.90	61.64	78.41	85.41	96.04	95.75	20.81	77.99	21.86
MSPT (Curvo et al., 2026)	9.28	19.15	23.18	25.89	28.14	29.94	27.70	61.18	73.90	86.65	101.71	110.88	23.42	79.14	24.39
Transformer$_{\text{galerkin}}$ (Cao, 2021)	11.89	19.49	23.07	28.43	35.67	42.74	38.61	69.13	96.66	115.08	136.97	168.67	26.86	104.64	28.11
DilResNet (Stachenfeld et al., 2022)	8.55	17.99	22.08	23.90	25.04	25.30	29.00	61.87	78.00	83.80	86.84	84.99	21.44	74.56	22.30
PhysGTO (Liu et al., 2026)	9.41	18.51	21.85	23.68	25.69	26.97	30.52	60.97	70.12	78.40	89.10	92.79	21.81	72.96	22.99
GNS (Sanchez-Gonzalez et al., 2020)	9.67	19.74	24.31	27.72	30.70	32.74	29.69	61.92	78.24	89.28	99.89	106.83	24.97	80.91	26.09
DeepONet (Lu et al., 2021)	16.68	21.74	26.20	28.05	28.29	28.67	60.07	73.64	86.37	91.34	92.49	90.63	25.51	84.65	26.38
GraphViT (Janny et al., 2023)	8.33	17.05	20.41	22.18	23.61	24.44	28.28	57.98	69.36	71.16	75.61	76.20	20.15	66.05	21.00
BSMS (Cao et al., 2023)	8.44	17.30	21.20	24.12	26.66	28.85	**27.01**	54.20	61.57	68.21	77.84	81.24	21.78	63.92	22.51
FNO (Li et al., 2021)	10.70	18.04	22.07	26.54	33.95	42.81	31.13	**52.02**	60.85	66.83	71.30	76.06	25.49	61.69	26.08
LinearNO (Hu et al., 2026b)	10.26	16.81	19.74	21.66	22.92	23.84	29.14	52.05	60.81	65.82	72.07	72.55	19.82	61.00	20.58
Transolver (Wu et al., 2024)	10.19	**16.24**	19.10	20.73	22.38	23.45	29.77	54.27	61.51	66.99	74.53	75.64	19.23	62.76	19.99
MWT (Gupta et al., 2021)	9.55	16.50	18.67	20.26	22.03	23.35	32.05	57.19	70.58	83.28	97.58	106.52	18.97	76.64	20.11
Persistence	14.14	20.78	19.80	20.42	20.92	21.35	40.17	60.19	54.16	54.63	58.75	58.21	20.14	56.09	20.95
DynaSolver (Ours)	11.61	16.27	**17.37**	**17.86**	**18.06**	**18.36**	34.02	53.19	**57.48**	**59.17**	**60.37**	**61.61**	**17.07**	**56.35**	**17.71**

4 EXPERIMENTS

We evaluate DynaSolver on RoboCFD-4D in terms of prediction accuracy, rollout stability, and efficiency, followed by controlled ablations on the model design and training strategy. Detailed training configurations and evaluation protocols are provided in the appendix (Sec. A)



(a) Overall rollout error. (b) Velocity rollout error. (c) Pressure rollout error.

Figure 4: **Rollout error evolution on RoboCFD-4D Base.** Relative L_2 errors are reported across the full autoregressive rollout for the overall flow field, velocity, and pressure.

4.1 ROBOCFD-4D DATASET

We introduce **RoboCFD-4D**, a dynamic-boundary 4D CFD benchmark for articulated robotic motion. Each trajectory contains 30 temporally ordered CFD states ($T_{\text{roll}} = 29$), each represented on an $18 \times 18 \times 36$ model grid ($N = 11{,}664$) with aligned volumetric and surface-associated features, and spans freestream velocities in $\{2, 3, 4, 6, 7, 8\}$ m/s under multiple yaw angles. **RoboCFD-4D Base**, used for the main experiments, contains 216 training trajectories from 18 robot morphologies and 136 development/evaluation trajectories covering unseen wind conditions and four unseen morphologies. **RoboCFD-4D Mini** is a strict subset used for controlled ablations, with 96 training trajectories from eight morphologies and 48 development/evaluation trajectories including one unseen morphology. Details of the simulation and dataset construction pipeline are provided in the appendix (Sec. B).

4.2 MAIN RESULTS

We compare DynaSolver with representative learning-based CFD surrogates on RoboCFD-4D Base. A non-learned persistence baseline is also included, which keeps the initial

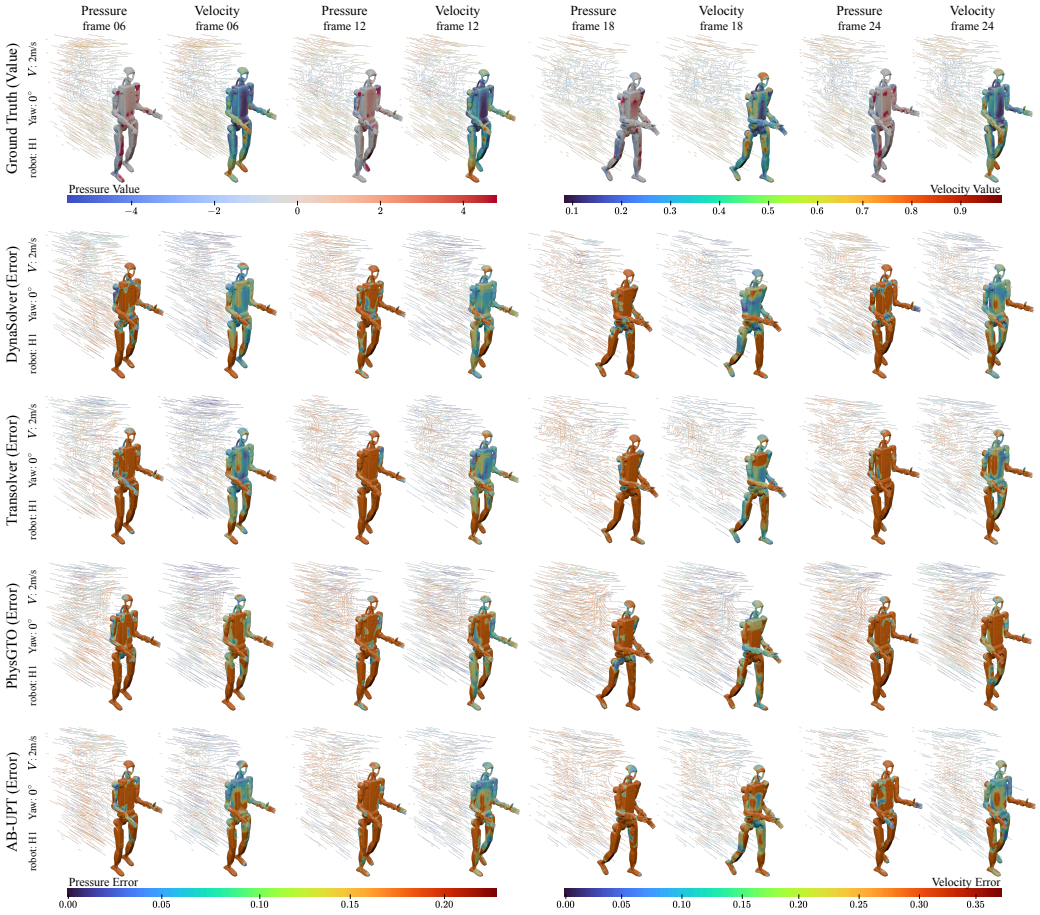


Figure 5: **Qualitative comparison across autoregressive rollout.** Ground-truth fields and corresponding error maps are shown at selected rollout steps for an unseen robot morphology. DynaSolver shows lower long-horizon errors than the baselines, particularly for velocity.

flow state unchanged throughout rollout, i.e., $\widehat{Y}_t = Y_0$, providing a no-evolution reference against which the value of modeling dynamic boundary and flow evolution can be assessed. As shown in Table 2, although DynaSolver does not attain the lowest one-step error, its advantage emerges during autoregressive rollout, where error grows substantially more slowly than for competing learned surrogates. From step 12 onward, it consistently achieves the lowest velocity and pressure errors among learned methods. At the final step $t = 29$, DynaSolver reaches 18.36% velocity error and 61.61% pressure error, compared with the best competing results of 23.35% and 72.55%, respectively, corresponding to relative reductions of 21.4% and 15.1%. The continuous rollout curves in Fig. 4 and qualitative comparisons in Fig. 5 further illustrate this behavior, with visibly lower prediction errors at later rollout steps, particularly for velocity.

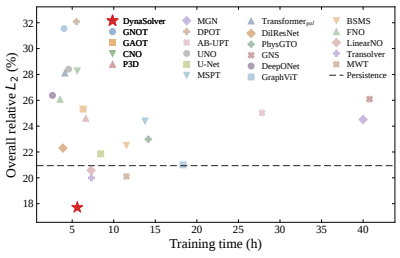


Figure 6: **Training cost versus overall rollout error.** Overall rollout relative L_2 error is plotted against training time; DynaSolver achieves the lowest rollout error while maintaining low training cost.

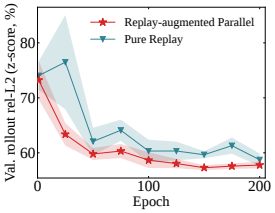
The advantage is also reflected in the full-rollout averages. DynaSolver achieves 17.07% velocity error, 56.35% pressure error, and 17.71% overall error, yielding an 11.4% relative reduction over the best competing overall result of 19.99%. Importantly, this accuracy does not come at the expense of training efficiency: Fig. 6 shows that DynaSolver attains the lowest overall rollout error while remaining among the most training-efficient learned methods. Together, these results demonstrate reduced autoregressive error accumulation with low training cost.

Table 3: **Progressive design ablation on RoboCFD-4D Mini.** We report the full-rollout relative L_2 errors (%) together with training cost, inference latency, and model size. The blue row is ours.

Variants	Average Velocity $L_2 \downarrow$	Average Pressure $L_2 \downarrow$	Overall $L_2 \downarrow$	Train Time (h) $\downarrow$	Inference Time (ms) $\downarrow$	Params
Transolver	20.12	71.39	21.06	<u>1.525</u>	**8.09**	3.87M
+ Boundary Evolution	19.83	66.45	20.69	**1.524**	<u>8.14</u>	3.87M
+ Causal Temporal Attention	17.53	45.91	17.98	6.678	14.79	5.98M
+ Surface-Volume Dual Branch	<u>16.24</u>	**42.97**	<u>16.68</u>	11.470	27.34	8.09M
+ Amortized Token-Space Operator	18.39	53.13	18.98	3.087	9.99	8.09M
+ Pointwise MLP Decoder	**15.65**	<u>44.30</u>	**16.14**	3.384	10.85	8.62M

Table 4: **Training-strategy ablation on RoboCFD-4D Mini.** Left: full-rollout relative L_2 errors (%) and training cost under the same DynaSolver architecture. Right: training convergence over epochs. The blue row is ours.

Training Strategy	Avg. Vel. $L_2 \downarrow$	Avg. Pre. $L_2 \downarrow$	Overall $L_2 \downarrow$	Training Time (h) $\downarrow$
Unroll	16.00	43.83	16.47	16.575
Pushforward	16.04	44.78	16.53	16.476
Parallel TF	17.15	65.08	18.18	**1.792**
Parallel TF + Noise	16.45	52.87	17.14	3.342
Pure Replay	<u>16.17</u>	<u>48.21</u>	<u>16.72</u>	<u>2.824</u>
Replay-Augmented Parallel	**15.65**	**44.30**	**16.14**	3.384



4.3 Ablation Studies

Progressive model design. Table 3 progressively builds DynaSolver from the spatial Transolver backbone. Boundary-evolution conditioning first improves sensitivity to moving-boundary dynamics, with the clearest gain in pressure prediction. Introducing causal temporal modeling then yields the largest improvement, reducing the overall rollout error from 20.69% to 17.98%, while the dual surface–volume branch further lowers it to 16.68%. We next move the repeated computation into the compact token space, which sharply reduces the training cost from 11.47 h to 3.09 h but sacrifices accuracy when used alone. The lightweight point decoder compensates for this lost point-level capacity, recovering the performance while retaining most of the efficiency gain; the complete DynaSolver achieves the best overall error of 16.14% with only 3.38 h of training.

Training strategy ablation. Table 4 fixes the complete DynaSolver architecture and isolates the effect of training strategy. Dense autoregressive schemes such as Unroll and Pushforward achieve strong rollout accuracy, but require costly sequential rollout during training. Parallel TF instead exploits block-causal attention to optimize all transitions in a clip simultaneously and is substantially faster, yet its 18.18% overall error reflects the mismatch between ground-truth-conditioned training and autoregressive rollout. Injecting Gaussian noise (Pfaff et al., 2021) partially alleviates this mismatch, reducing the error to 17.14%, while Pure Replay further improves it to 16.72% by exposing the model to autoregressively generated states. Replay-Augmented Parallel Training combines replay histories with clean ground-truth histories while retaining parallel optimization. It achieves the lowest overall error of 16.14% in 3.38 h, outperforming both Unroll (16.47%) and Pushforward (16.53%) while reducing training time by approximately $4.9\times$. The convergence curves in Table 4 further show that gradually introducing replay yields more stable optimization and lower rollout error than Pure Replay.

5 Conclusion

We presented DynaSolver for 4D CFD prediction with continuously evolving robot boundaries. DynaSolver models causal spatio-temporal flow evolution under dynamic boundaries through an efficient physical-token temporal operator, while Replay-Augmented Parallel Training exposes the model to autoregressive histories while preserving efficient parallel optimization. Experiments on RoboCFD-4D show that DynaSolver achieves strong prediction accuracy and training efficiency, with its advantage becoming increasingly pronounced as rollout proceeds. These results demonstrate reduced autoregressive error accumulation and support causal spatio-temporal modeling as an effective approach to neural CFD prediction under dynamic geometry.

AI USE STATEMENT

Generative AI tools were used for limited assistance with manuscript editing, reference search and formatting, and visual refinement of qualitative figures. All scientific content, including the dataset, simulation pipeline, method, experimental design, numerical results, and analysis, was produced by the authors.

All AI-assisted material was manually reviewed and verified. Figures were constructed from the authors' simulation outputs, references were checked against the original publications or official records, and AI-assisted text was validated against the underlying experiments. The authors take full responsibility for the final manuscript and all reported results.

ETHICS STATEMENT

RoboCFD-4D is generated entirely in simulation and contains no human subjects, personal data, or real-world user data. The articulated robot geometries are derived from publicly available assets, and their provenance and licensing information will be documented in the released dataset card. Assets without verifiable provenance or compatible licensing are excluded.

The benchmark targets simulated CFD prediction around prescribed articulated motion and is not intended as a validated surrogate for safety-critical robotic or aerodynamic deployment. Its operating conditions, spatial resolution, and simulated dynamics cover only the regimes studied in this work. We will release the benchmark and associated metadata under an appropriate research license together with procedures for reporting provenance or licensing concerns.

REPRODUCIBILITY STATEMENT

The model formulation, architecture, training objective, and evaluation protocol are specified in the main paper and appendix. Section 3 defines the dynamic-boundary prediction problem, the physical-token temporal operator, and Replay-Augmented Parallel Training. Section A reports the model hyperparameters, training schedule, replay configuration, rollout protocol, evaluation metrics, and hardware used in our experiments.

The construction of RoboCFD-4D is documented in Section B. We will release the benchmark dataset together with the metadata and split information required to reproduce the reported evaluation protocol.

We will additionally release the training and evaluation code, model checkpoints, and scripts used to reproduce the main tables and figures. Where redistribution of a source robot asset is restricted by its original license, we will provide its provenance and corresponding processing configuration rather than redistributing the asset itself.

REFERENCES

Benedikt Alkin, Maurits Bleeker, Richard Kurle, Tobias Kronlachner, Reinhard Sonnleitner, Matthias Dorfer, and Johannes Brandstetter. AB-UPT: scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. *Trans. Mach. Learn. Res.*, 2025, 2025.

Neil Ashton, Danielle C. Maddix, Samuel Gundry, and Parisa M. Shabestari. Ahmedml: High-fidelity computational fluid dynamics dataset for incompressible, low-speed bluff body aerodynamics. *arXiv preprint arXiv:2407.20801*, 2024a.

Neil Ashton, Charles Mockett, Marian Fuchs, Louis Fliessbach, Hendrik Hetmann, Thilo Knacke, Norbert Schonwald, Vangelis Skaperdas, Grigoris Fotiadis, Astrid Walle, Burkhard Hupertz, and Danielle C. Maddix. Drivaerml: High-fidelity computational fluid dynamics dataset for road-car external aerodynamics. *arXiv preprint arXiv:2408.11969*, 2024b.

Ibrahim Ayed, Emmanuel de Bézenac, Arthur Pajot, Julien Brajard, and Patrick Gallinari. Learning dynamical systems from partial observations. *arXiv preprint arXiv:1902.11136*, 2019.

Giacomo Baldan and Alberto Guardone. A deep neural network reduced order model for unsteady aerodynamics of pitching airfoils. *Aerospace Science and Technology*, 152:109345, 2024.

Philipp Bekemeyer, Nathan S. Hariharan, Andrew M. Wissink, and Jason Cornelius. Introduction of applied aerodynamics surrogate modeling benchmark cases. *AIAA SCITECH 2025 Forum*, 2025.

Florent Bonnet, Jocelyn Ahmed Mazari, Paola Cinnella, and Patrick Gallinari. Airfrans: High fidelity computational fluid dynamics dataset for approximating reynolds-averaged navier-stokes solutions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Johannes Brandstetter, Daniel Worrall, and Max Welling. Message passing neural PDE solvers. In *International Conference on Learning Representations (ICLR)*, 2022.

Shuhao Cao. Choose a transformer: Fourier or galerkin. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 24924–24940, 2021.

Yadi Cao, Menglei Chai, Minchen Li, and Chenfanfu Jiang. Efficient learning of mesh-based physical simulation with bi-stride multi-scale graph neural network. In *International Conference on Machine Learning, ICML*, volume 202, pp. 3541–3558. PMLR, 2023.

Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and what's next. *J. Sci. Comput.*, 92(3):88, 2022.

Pedro M. P. Curvo, Jan-Willem van de Meent, and Maksim Zhdanov. MSPT: Efficient large-scale physical modeling via parallelized multi-scale attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12924–12933, 2026.

Filipe de Avila Belbute-Peres, Thomas D. Economon, and J. Zico Kolter. Combining differentiable PDE solvers and graph neural networks for fluid flow prediction. In *International Conference on Machine Learning, ICML*, volume 119, pp. 2402–2411. PMLR, 2020.

Jingyang Deng, Xingjian Li, Haoyi Xiong, Xiaoguang Hu, and Jinwen Ma. Geometry-guided conditional adaptation for surrogate models of large-scale 3d PDEs on arbitrary geometries. In *Proceedings of the International Joint Conference on Artificial Intelligence, IJCAI*, pp. 5790–5798, 2024.

Tao Du, Josie Hughes, Sebastien Wah, Wojciech Matusik, and Daniela Rus. Underwater soft robot modeling and control with differentiable simulation. *IEEE Robotics and Automation Letters*, 6 (3):4994–5001, 2021.

Mohamed Elrefaie, Florin Morar, Angela Dai, and Faez Ahmed. Drivaernet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Mohamed Elrefaie, Angela Dai, and Faez Ahmed. Drivaernet: A parametric car dataset for data-driven aerodynamic design and prediction. *Journal of Mechanical Design*, 147(4), 2025.

Gaurav Gupta, Xiongye Xiao, and Paul Bogdan. Multiwavelet-based operator learning for differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pp. 24048–24062, 2021.

Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, and Jun Zhu. GNOT: A general neural operator transformer for operator learning. In *International Conference on Machine Learning ICML*, volume 202, pp. 12556–12569. PMLR, 2023.

Zhongkai Hao, Chang Su, Songming Liu, Julius Berner, Chengyang Ying, Hang Su, Anima Anandkumar, Jian Song, and Jun Zhu. DPOT: auto-regressive denoising operator transformer for large-scale PDE pre-training. In *International Conference on Machine Learning ICML*, volume 235, pp. 17616–17635. PMLR / OpenReview.net, 2024.

Benjamin Holzschuh, Georg Kohl, Florian Redinger, and Nils Thuerey. P3D: Highly scalable 3d neural surrogates for physics simulations with global context. In *International Conference on Learning Representations (ICLR)*, 2026.

Peiyan Hu, Rui Wang, Xiang Zheng, Tao Zhang, Haodong Feng, Ruiqi Feng, Long Wei, Yue Wang, Zhi-Ming Ma, and Tailin Wu. Wavelet diffusion neural operator. In *International Conference on Learning Representations, ICLR*. OpenReview.net, 2025.

Peiyan Hu, Haodong Feng, Hongyuan Liu, Tongtong Yan, Wenhao Deng, Tianrun Gao, Rong Zheng, Haoren Zheng, Chenglei Yu, Chuanrui Wang, Kaiwen Li, Zhi-Ming Ma, Dezhi Zhou, Xingcai Lu, Dixia Fan, and Tailin Wu. Realpdebench: A benchmark for complex physical systems with real-world data. In *International Conference on Learning Representations (ICLR)*, 2026a.

Wenjie Hu, Sidun Liu, Peng Qiao, Zhenglun Sun, and Yong Dou. Transolver is a linear transformer: Revisiting physics-attention through the lens of linear attention. In *Proceedings of the AAAI Conference on Artificial Intelligence AAAI*, pp. 408–416, 2026b.

Yuanming Hu, Jiancheng Liu, Andrew Spielberg, Joshua B. Tenenbaum, William T. Freeman, Jiajun Wu, Daniela Rus, and Wojciech Matusik. Chainqueen: A real-time differentiable physical simulator for soft robotics. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6265–6271, 2019.

Steeven Janny, Aurélien Benetteau, Nicolas Thome, Madiha Nadri, Julie Digne, and Christian Wolf. Eagle: Large-scale learning of turbulent fluid dynamics with mesh transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

Anya R. Jones, Oksan Cetiner, and Marilyn J. Smith. Physics and modeling of large flow disturbances: Discrete gust encounters for modern air vehicles. *Annual Review of Fluid Mechanics*, 54: 469–493, 2022.

Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021.

Sahin Lale, Peter I. Renn, Kamyar Azizzadenesheli, Babak Hassibi, Morteza Gharib, and Anima Anandkumar. Falcon: Fourier adaptive learning and control for disturbance rejection under extreme turbulence. *npj Robotics*, 2, 2024.

Zongyi Li, Nikola B. Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020.

Zongyi Li, Nikola Borislavov Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations, ICLR*, 2021.

Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for pdes on general geometries. *J. Mach. Learn. Res.*, 24:388:1–388:26, 2023a.

Zongyi Li, Nikola B. Kovachki, Christopher B. Choy, Boyi Li, Jean Kossaifi, Shourya Prakash Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, and Animashree Anandkumar. Geometry-informed neural operator for large-scale 3d pdes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023b.

Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/IMS Journal of Data Science*, 1(3):1–27, 2024.

Bjoern List, Li-Wei Chen, Kartik Bali, and Nils Thuerey. Differentiability in unrolled training of neural physics simulators on transient dynamics. *Computer Methods in Applied Mechanics and Engineering*, 433:117441, 2025.

Pengwei Liu, Xingyu Ren, Pengkai Wang, Hangjie Yuan, Zhongkai Hao, Guanyu Chen, Chao Xu, Dong Ni, and Shengze Cai. An efficient graph-transformer operator for learning physical dynamics with manifolds embedding. *Communications AI & Computing*, 1:9, 2026.

Antonio Loquercio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Learning high-speed flight in the wild. *Science Robotics*, 6(59):eabg5810, 2021.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.

Yining Luo, Yingfa Chen, and Zhen Zhang. Cfdbench: A large-scale benchmark for machine learning methods in fluid dynamics. *arXiv preprint arXiv:2310.05963*, 2023.

Antonello Paolino, Gabriele Nava, Fabio Di Natale, Fabio Bergonti, Punith Reddy Vanteddu, Donato Grassi, Luca Riccobene, Alex Zanotti, Renato Tognaccini, Gianluca Iaccarino, and Daniele Pucci. Learning aerodynamics for the control of flying humanoid robots. *Communications Engineering*, 4(1):111, 2025.

Eric J Parish and Karthik Duraisamy. Non-markovian closure models for large eddy simulations using the mori-zwanzig formalism. *Physical Review Fluids*, 2(1):014604, 2017.

Charles S Peskin. The immersed boundary method. *Acta numerica*, 11:479–517, 2002.

Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations (ICLR)*, 2021.

Md Ashiqur Rahman, Zachary E. Ross, and Kamyar Azizzadenesheli. U-NO: u-shaped neural operators. *Trans. Mach. Learn. Res.*, 2023, 2023.

Bogdan Raonic, Roberto Molinaro, Tim De Ryck, Tobias Rohner, Francesca Bartolucci, Rima Alaifari, Siddhartha Mishra, and Emmanuel de Bézenac. Convolutional neural operators for robust and accurate learning of pdes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI) 2015*, volume 9351, pp. 234–241. Springer, 2015.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 119, pp. 8459–8468, 2020.

Fotis Sotiropoulos and Xiaolei Yang. Immersed boundary methods for simulating fluid–structure interaction. *Progress in Aerospace Sciences*, 65:1–21, 2014.

Kimberly Stachenfeld, Drummond B. Fielding, Dmitrii Kochkov, Miles Cranmer, Tobias Pfaff, Jonathan Godwin, Can Cui, Shirley Ho, Peter Battaglia, and Alvaro Sanchez-Gonzalez. Learned coarse models for efficient turbulence simulation. In *International Conference on Learning Representations (ICLR)*, 2022.

Alasdair Tran, Alexander Patrick Mathews, Lexing Xie, and Cheng Soon Ong. Factorized fourier neural operators. In *International Conference on Learning Representations, ICLR*. OpenReview.net, 2023.

Nobuyuki Umetani and Bernd Bickel. Learning three-dimensional flow for interactive aerodynamic design. *ACM Trans. Graph.*, 37(4):89, 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5998–6008, 2017.

Gege Wen, Zongyi Li, Kamyar Azizzadenesheli, Anima Anandkumar, and Sally M. Benson. U-FNO—an enhanced fourier neural operator-based deep-learning model for multiphase flow. *Advances in Water Resources*, 163:104180, 2022.

Shizheng Wen, Arsh Kumbhat, Levi Lingsch, Sepehr Mousavi, Yizhou Zhao, Praveen Chandrashekar, and Siddhartha Mishra. Geometry aware operator transformer as an efficient and accurate neural surrogate for pdes on arbitrary domains. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, Main Conference, pp. 155423–155501. Curran Associates, Inc., 2025.

Gabriel D. Weymouth and Bernat Font. Waterlily.jl: A differentiable and backend-agnostic julia solver for incompressible viscous flow around dynamic bodies. *Computer Physics Communications*, 315:109748, 2025.

Haixu Wu, Tengge Hu, Huakun Luo, Jianmin Wang, and Mingsheng Long. Solving high-dimensional pdes with latent spectral models. In *International Conference on Machine Learning ICML*, volume 202, pp. 37417–37438. PMLR, 2023.

Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 235, pp. 53681–53705. PMLR, 2024.

Chao Xia, Carl Lewerth, Xi Zeng, Alexey Vdovin, Simone Sebben, and Qing Jia. Drivaerrim: A computational fluid dynamics dataset for automotive rim aerodynamics, 2026.

Wenzhuo Xu, Noelia Grande Gutiérrez, and Christopher McComb. Megaflow2d: A parametric dataset for machine learning super-resolution in computational fluid dynamics simulations. In *Proceedings of Cyber-Physical Systems and Internet of Things (CPS-IoT) Week Workshops*, pp. 100–104. ACM, 2023.

Hang Zhou, Haixu Wu, Haonan Shangguan, Yuezhou Ma, Huikun Weng, Jianmin Wang, and Mingsheng Long. Transolver-3: Scaling up transformer solvers to industrial-scale geometries. *arXiv preprint arXiv:2602.04940*, 2026.

A EXPERIMENTAL SETUP

Input representation. For each transition from $t-1$ to t, DynaSolver receives the spatial condition $c_t \in \mathbb{R}^{N\times 11}$, the boundary-evolution feature $b_{t-1\to t} \in \mathbb{R}^{N\times 8}$, and the previous CFD state $Y_{t-1} \in \mathbb{R}^{N\times 4}$. Table 5 summarizes the channel definitions. All features are defined pointwise on the model grid, with surface-associated attributes provided at the corresponding projected grid nodes. The signed-distance convention is $\phi > 0$ in the fluid domain and $\phi < 0$ inside the solid, and surface normals point outward from the robot. All baselines receive the spatial condition c_t and previous CFD state Y_{t-1}. DynaSolver additionally incorporates the proposed boundary-evolution representation $b_{t-1\to t}$; its contribution is isolated in the progressive model ablation in Table 3.

Table 5: **Pointwise input features used by DynaSolver.** S and V denote surface-associated and volume grid nodes, respectively.

Input	Feature	Dim.	Samples	Representation
c_t	normalized position $\widetilde{\mathbf{x}}_t$	3	S/V	dimensionless
	signed distance ϕ_t	1	S/V	m
	nearest outward normal $\mathbf{n}_t$	3	S/V	unit vector
	inflow direction $\mathbf{d}_\infty$	3	S/V	unit vector
	normalized inflow speed $\widetilde{U}_\infty$	1	S/V	dimensionless
$b_{t-1\to t}$	previous signed distance ϕ_{t-1}	1	S/V	m
	signed-distance change $\phi_t - \phi_{t-1}$	1	S/V	m
	wall velocity $\mathbf{v}^{\mathrm{w}}_{t-1\to t}$	3	S/V	m/s
	fluid indicators (χ_{t-1}, χ_t)	2	S/V	binary
	normalized time interval $\widetilde{\Delta t}$	1	S/V	dimensionless

Model and training configuration. Our default DynaSolver uses a latent width of $D = 256$, $L = 8$ operator blocks with eight attention heads, $G = 16$ physics tokens per surface/volume branch, and a causal window of $W = 4$. We sample non-overlapping temporal clips with stride 4 and train all models from scratch for 200 epochs with batch size 4, learning rate 10^{-4}, and weight decay 10^{-5}. For Replay-Augmented Parallel Training, the replay probability ρ is gradually increased from 0 to 0.5 over the first 20 epochs and kept fixed thereafter. All reported training and inference times are measured on a single NVIDIA A800 GPU. Base is used for the main comparison, while Mini is used for controlled ablations under the same model and optimization configuration.

Evaluation protocol and metrics. Following the formulation in Sec. 3.1, we evaluate all models under closed-loop autoregressive rollout over the full horizon T_{roll}, starting from the given initial state Y_0 and recursively feeding each prediction back into the subsequent causal context.

The CFD state is $Y_t = [u, v, w, p]_t$, where $\mathbf{u} = (u, v, w)$ denotes the velocity vector and p denotes gauge-referenced kinematic pressure. For each trajectory, let U_∞ denote the magnitude of the prescribed freestream velocity. We first nondimensionalize the flow variables as

$$\mathbf{u}^* = \frac{\mathbf{u}}{U_\infty}, \qquad p^* = \frac{p}{U_\infty^2}, \tag{9}$$

where $(\cdot)^*$ denotes the corresponding nondimensional quantity. Each channel is then standardized using its mean and standard deviation computed exclusively from the training split. Denoting this standardization by $\mathcal{N}(\cdot)$, the model is trained and autoregressively rolled out in the standardized space,

$$Z_t = \mathcal{N}(Y_t^*). \tag{10}$$

For evaluation, predicted states are first transformed back to the nondimensional space as $\widehat{Y}_t^* = \mathcal{N}^{-1}(\widehat{Z}_t)$. For a channel subset $\mathcal{S}$, the masked relative L_2 error is

$$E_{\mathcal{S}}(t) = \sqrt{\frac{\sum_{n=1}^{N} m_{t,n} \sum_{c\in\mathcal{S}} \left(\widehat{Y}^*_{t,n,c} - Y^*_{t,n,c}\right)^2}{\sum_{n=1}^{N} m_{t,n} \sum_{c\in\mathcal{S}} \left(Y^*_{t,n,c}\right)^2}}. \tag{11}$$

where $m_{t,n}$ excludes invalid samples from evaluation, including volume samples occupied by the moving solid according to M_t^{v}. We report velocity over $\mathcal{S}_v = \{u, v, w\}$, pressure over $\mathcal{S}_p = \{p\}$,

and the overall field over $\mathcal{S}_{\text{all}} = \{u, v, w, p\}$, with all errors expressed as percentages. Computing the metric in nondimensional space removes dependence on training-set standardization and avoids scale bias across variables.

Horizon-wise results report $E_{\mathcal{S}}(t)$ at selected rollout steps. The full-rollout score first averages the per-step errors over T_{roll} within each trajectory and then across trajectories. RoboCFD-4D uses a train/test protocol without a separate validation split. No checkpoint selection is performed on the test set; all models are trained for the fixed 200-epoch schedule described above, and the final-epoch checkpoint is used for evaluation. The same training and evaluation protocol is applied to all compared methods.

B RoboCFD-4D

We generate RoboCFD-4D by simulating flow around articulated robots undergoing prescribed motion. For each robot, a fixed 1 s motion sequence is evaluated under 16 prescribed wind conditions, with 200 raw CFD output frames stored for each completed condition. After removing the initial moving-boundary start-up transient, we construct temporally subsampled learning sequences from these outputs. Each retained frame contains both Eulerian flow fields and observations associated with a fixed set of points on the moving robot. Throughout simulation and export, versioned manifests and stable link indices preserve a consistent correspondence among geometry, motion, and flow observations throughout the dataset.

Public assets and link preparation. We obtain robot descriptions and meshes from public repositories, deduplicate overlapping descriptions, and convert URDF and MJCF assets to a common URDF-based articulation representation. This representation records the kinematic tree, joint axes and limits, and the geometry of each link in its local coordinate frame. After resolving mesh dependencies, we validate each link mesh independently. Meshes that fail geometry checks are repaired at the link level, with CGAL Alpha Wrap used where needed. Repairing geometry in the original link frame preserves its attachment to the kinematic tree and its correspondence with the subsequent link-local SDF representation. Before simulation, the resulting CFD inputs are checked for watertightness, winding consistency, and geometric validity.

Kinematic motion synthesis. We use two forms of prescribed kinematic motion. Most articulated joints are handled by a generic morphology-aware synthesis pipeline, whereas robot-specific profiles are used for mechanisms that require dedicated treatment, including continuous wheel rotations. For joints handled by the generic pipeline, motion follows the harmonic template

$$q_j(t) = \bar{q}_j + a_j \sin(2\pi f t + \phi_j) + b_j \cos(2\pi f t + \phi_j), \tag{12}$$

where $\bar{q}_j$ denotes the neutral pose, f is the motion frequency, and a_j, b_j, and ϕ_j control the excursion and relative timing of joint j. Generic-motion amplitudes are selected to satisfy joint position and velocity limits, followed by kinematic consistency checks. The root undergoes no net horizontal locomotion, although bounded lateral, vertical, and angular oscillations may be prescribed. The resulting trajectories provide smooth, kinematically valid articulated boundary motion rather than dynamically simulated locomotion.

Each source motion is sampled at $1\,\text{ms}$ intervals over $[0, 1]\,\text{s}$, yielding 1001 source samples. Forward kinematics provides the transform $\mathbf{T}_\ell(t) \in \text{SE}(3)$ of each link ℓ. Raw CFD outputs are stored every fifth source interval, at $t = 0.005, 0.010, \ldots, 1.000\,\text{s}$, yielding 200 frames per completed condition. Generic motion packages undergo joint-limit, temporal-closure, rotation, forward-kinematics, foot-clearance, and discrete bounding-box checks where applicable. Dedicated prescribed motion packages are audited separately for temporal and articulation compatibility with the CFD pipeline.

Dynamic geometry reconstruction. To provide the moving geometry to the flow solver, we construct a truncated signed distance field (SDF) ϕ_ℓ for each repaired link in its local coordinate system. The transformed link-local fields are combined into the composite implicit representation

$$\phi(\mathbf{x}, t) = \min_\ell \phi_\ell \left(\mathbf{T}_\ell^{-1}(t)\mathbf{x} \right), \tag{13}$$

where $\mathbf{x}$ is a world-space point, homogeneous coordinates are implicit, and negative values indicate solid interiors. The negative region of the composite field represents the union of the transformed

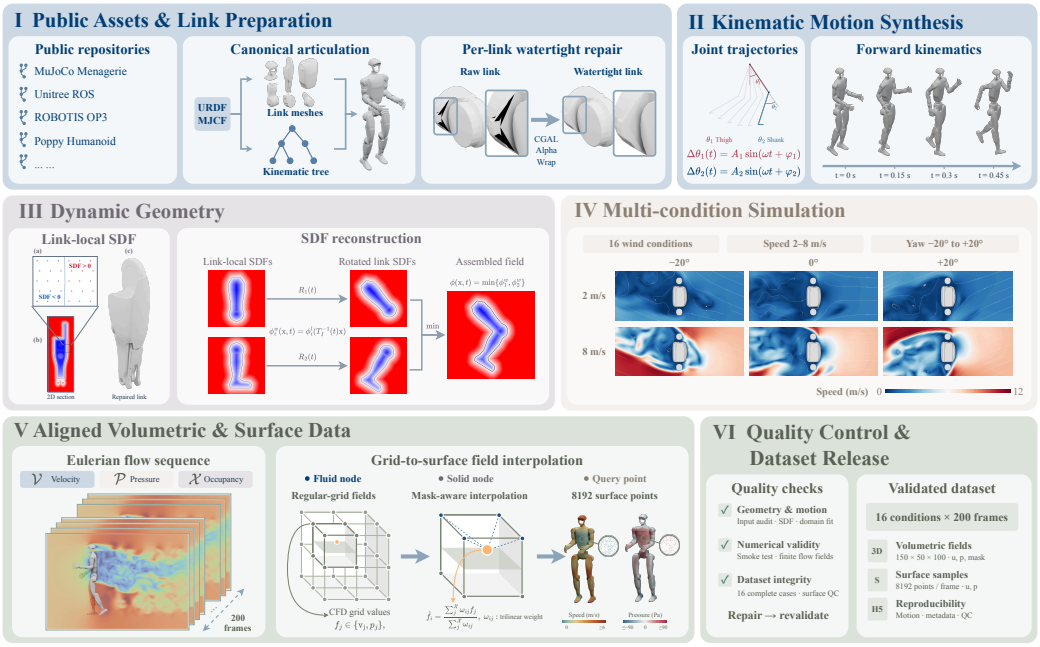


Figure 7: Overview of the RoboCFD-4D data generation pipeline. Public robot assets are standardized and repaired at the link level, followed by prescribed motion synthesis and reusable link-local SDF construction. The transformed link-local fields define the prescribed moving immersed boundary for multi-condition CFD simulations. Aligned volumetric fields and tracked near-surface observations are then extracted and subjected to geometric, numerical, and dataset-level validation before inclusion in the final published dataset.

link solids. The pointwise minimum is not required to remain an exact signed-distance function everywhere, particularly near link interfaces.

Small overlaps may occur between tightly fitted neighboring parts due to finite precision and discretization in the source meshes. Under the pointwise minimum, such overlaps are treated as part of a single solid union rather than as separate internal boundaries. They therefore reflect mesh precision at close-fitting interfaces rather than intended interpenetration between distinct robot components.

Each link-local SDF is precomputed once and reused throughout the sequence. At runtime, only the rigid link transforms are updated, and the composite field is evaluated from the transformed link-local fields using Eq. 13. This avoids reconstructing a volumetric geometry representation at every frame while preserving link-level geometry and articulation.

Each link-local SDF is validated using surface reconstruction error, inside–outside sign accuracy, and voxelized-volume agreement. Fields that fail these gates are rebuilt and revalidated before use. At representative poses, we additionally check link transforms and per-link mesh–SDF consistency. Separate domain-fit checks verify that the swept moving geometry remains inside the prescribed computational domain throughout the simulated motion.

Multi-condition flow simulation. We use WaterLily (Weymouth & Font, 2025) to solve the incompressible Navier–Stokes equations, with the articulated robot geometry defining a prescribed moving immersed boundary throughout the simulation:

$$\nabla \cdot \mathbf{u} = 0, \tag{14}$$

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u}, \tag{15}$$

Table 6: Wind conditions and repository identifiers used in RoboCFD-4D.

Speed (m/s)	Yaw	Condition ID
2	$-20°$	`u2_yaw_m20`
2	$0°$	`u2_yaw_p0`
2	$20°$	`u2_yaw_p20`
3	$-10°$	`u3_yaw_m10`
3	$10°$	`u3_yaw_p10`
4	$-20°$	`u4_yaw_m20`
4	$0°$	`u4_yaw_p0`
4	$20°$	`u4_yaw_p20`
6	$-20°$	`u6_yaw_m20`
6	$0°$	`u6_yaw_p0`
6	$20°$	`u6_yaw_p20`
7	$-10°$	`u7_yaw_m10`
7	$10°$	`u7_yaw_p10`
8	$-20°$	`u8_yaw_m20`
8	$0°$	`u8_yaw_p0`
8	$20°$	`u8_yaw_p20`

where $\mathbf{u}$ is velocity, p is kinematic pressure, and ν is kinematic viscosity. The composite articulated field and prescribed link motion determine the moving solid boundary throughout the simulation; no two-way fluid–structure coupling is used.

The 16 prescribed wind conditions are listed in Table 6.

All simulations use a $150 \times 50 \times 100$ grid with isotropic physical cell spacing. Grid dimensions are fixed across robots, whereas the physical cell size Δx is selected separately for each robot to accommodate its swept geometry over the prescribed motion and wind orientations. Thus, simulations share the same grid dimensions but not necessarily the same physical spatial resolution or domain size. Across the dataset, the swept vertical extent spans approximately 38–77 grid cells.

Before each full run, a short preliminary simulation checks geometry–solver integration and basic numerical consistency. During the dynamic segment, each wind condition uses a fixed time step selected from the grid spacing and motion-speed estimates. CFL ratios are monitored during execution, and normalized velocity divergence and flux imbalance are evaluated as part of the numerical quality-control procedure described below.

Aligned volumetric and near-surface observations. Each completed simulation produces 200 raw CFD output frames indexed from 0 to 199. Raw frame k corresponds to $t = (k + 1) \times 0.005\,\mathrm{s}$, so frame 0 is stored at $t = 0.005\,\mathrm{s}$. The earliest outputs immediately follow the transition from the stationary-body flow initialization to the prescribed moving-body simulation and therefore contain a pronounced start-up transient.

To reduce the influence of this transition regime on the learning benchmark, we discard raw frames 0–19 and uniformly subsample the remaining 180 frames with a stride of 6. The retained zero-based raw-frame indices are

$$20, 26, 32, \ldots, 194,$$

corresponding to

$$t = 0.105, 0.135, 0.165, \ldots, 0.975\,\mathrm{s}.$$

This yields 30 temporally ordered frames per sequence, or 29 one-step transitions for autoregressive learning. The same temporal indices are applied to all volumetric fields, link transforms, tracked surface points, and near-surface observations, thereby ensuring consistent temporal alignment across all exported data modalities throughout each learning sequence.

For each raw CFD output, we store the cell-centered velocity vector, speed magnitude, gauge-normalized kinematic pressure, and fluid mask on the Eulerian grid. We denote the fluid mask by χ, with $\chi = 1$ in the fluid and $\chi = 0$ in the solid. Velocity and pressure values in solid cells are stored as NaN for downstream processing.

Pressure is reported under a fixed gauge convention. At each output frame, we subtract from the full pressure field the mean pressure over a predefined set of valid fluid probes near the outlet side of the wind-aligned simulation grid. This operation fixes only the additive pressure reference; it is not imposed as a physical outlet-pressure boundary condition.

The gauge-probe candidates lie on the last two x-planes of the wind-aligned simulation grid. Along the y and z directions, candidate grid points are selected starting from the first cell with a stride of four cells. For the $150 \times 50 \times 100$ grid, this yields at most

$$2 \times 13 \times 25 = 650$$

candidate probes per frame. Only candidates classified as fluid by the current-frame fluid mask are retained. Let $\mathcal{P}_t$ denote this valid probe set. The reported gauge-normalized kinematic pressure is

$$p_g(\mathbf{x}, t) = p(\mathbf{x}, t) - \frac{1}{|\mathcal{P}_t|} \sum_{\mathbf{x}_j \in \mathcal{P}_t} p(\mathbf{x}_j, t). \tag{16}$$

Export fails if $\mathcal{P}_t$ is empty. The residual mean of p_g over $\mathcal{P}_t$ is subsequently required to be no greater than 10^{-4} in the stored kinematic-pressure units. Probe locations are fixed in the wind-aligned simulation-grid frame and therefore do not correspond to identical world-frame locations across different yaw conditions. We do not assume that the probe pressure equals ambient pressure or that the far-field pressure is zero. The stored volumetric pressure field p_g has units of $\mathrm{m}^2\,\mathrm{s}^{-2}$.

We additionally construct a deterministic bank of 8192 tracked surface points for each robot. Each link receives a minimum allocation, with the remaining points distributed according to link surface area. The local coordinates and link identities of these points are fixed once and reused across all frames and wind conditions. Surface point i therefore follows its associated link according to

$$\mathbf{x}_i(t) = \mathbf{T}_{\ell(i)}(t)\mathbf{x}_i^{\mathrm{local}}, \tag{17}$$

where $\ell(i)$ denotes the link associated with point i. Because both $\mathbf{x}_i^{\mathrm{local}}$ and $\ell(i)$ remain fixed, a point index identifies the same material location throughout the motion sequence and across wind conditions. This construction provides deterministic material-point correspondence while retaining a fixed surface-array layout. We also retain transformed surface normals and material-point velocities obtained from rigid-body kinematics under the corresponding link motion.

Flow observations are transferred from the regular grid to the tracked surface points using mask-aware trilinear interpolation. For each tracked surface point with outward normal $\mathbf{n}_i(t)$, the corresponding external flow field is queried at

$$\mathbf{x}_i^{\mathrm{query}}(t) = \mathbf{x}_i(t) + 0.5\,\Delta x\,\mathbf{n}_i(t), \tag{18}$$

where Δx is the scalar spacing of the isotropic, cell-centered simulation grid. The half-cell displacement is a fixed, reproducible sampling convention that places the query on the exterior side of the moving surface. It is not an optimized offset, and we did not perform a dedicated offset ablation to establish $0.5\,\Delta x$ as optimal.

After mapping the query position to the cell-centered simulation grid, a scalar field or velocity component f is interpolated as

$$\widehat{f}_i(t) = \frac{\sum_{j \in \mathcal{N}_i(t)} m_j(t) w_{ij}(t) f_j(t)}{\sum_{j \in \mathcal{N}_i(t)} m_j(t) w_{ij}(t)}, \tag{19}$$

where $\mathcal{N}_i(t)$ contains the eight neighboring cell-centered grid samples, $w_{ij}(t)$ are the corresponding trilinear weights, and $m_j(t) = 1$ for finite fluid samples and 0 otherwise. Solid or non-finite neighbors are excluded from both sums, and the remaining weights are renormalized. A query is marked invalid and stored as NaN if the denominator in Eq. 19 contains no positive valid-fluid weight. For quality control, we retain both an interpolation-validity mask and the number of valid fluid corners in the local stencil.

These quantities are near-surface external-flow observations rather than exact wall boundary values. Flow velocity is stored separately from material-point velocity, and near-surface flow vectors are expressed in world coordinates. Near-surface pressure is sampled from the gauge-normalized kinematic pressure p_g and is also provided in pascals using

$$p_{\mathrm{Pa}} = \rho p_g, \tag{20}$$

with $\rho = 1.225\,\mathrm{kg\,m}^{-3}$.

Model-grid construction for learning. The raw Eulerian field contains $150 \times 50 \times 100 = 750{,}000$ cell-centered samples per frame. For model training and evaluation, we do not randomly or importance-sample a subset of these cells. Instead, for each CFD condition we construct a deterministic, cell-centered $18 \times 18 \times 36$ grid over the physical bounds of the corresponding simulation domain and resample the Eulerian velocity, pressure, and fluid mask onto this grid using trilinear interpolation. This produces exactly

$$N = 18 \times 18 \times 36 = 11{,}664$$

model nodes per frame. The coarse spatial grid is fixed for all retained frames of a given sequence, while its physical coordinates are inherited from the corresponding CFD domain; consequently, different robots or wind conditions are not assumed to share identical world-space node coordinates. The same deterministic preprocessing is used for both training and evaluation, and the current benchmark configuration uses all $11{,}664$ nodes rather than resampling a different point subset at each frame or evaluation pass.

The 8192 tracked near-surface samples described above are not appended to these $11{,}664$ nodes as additional model points. Instead, valid tracked surface samples are projected onto cells of the same $18 \times 18 \times 36$ model grid, where they provide the surface mask, surface normals, material velocity, and the corresponding surface-associated loss weights. Multiple tracked samples may map to the same coarse-grid cell and therefore collapse onto a single model node. Thus, $N = 11{,}664$ denotes the complete structured model grid, not a concatenation of 8192 surface points with an independently sampled set of volume points.

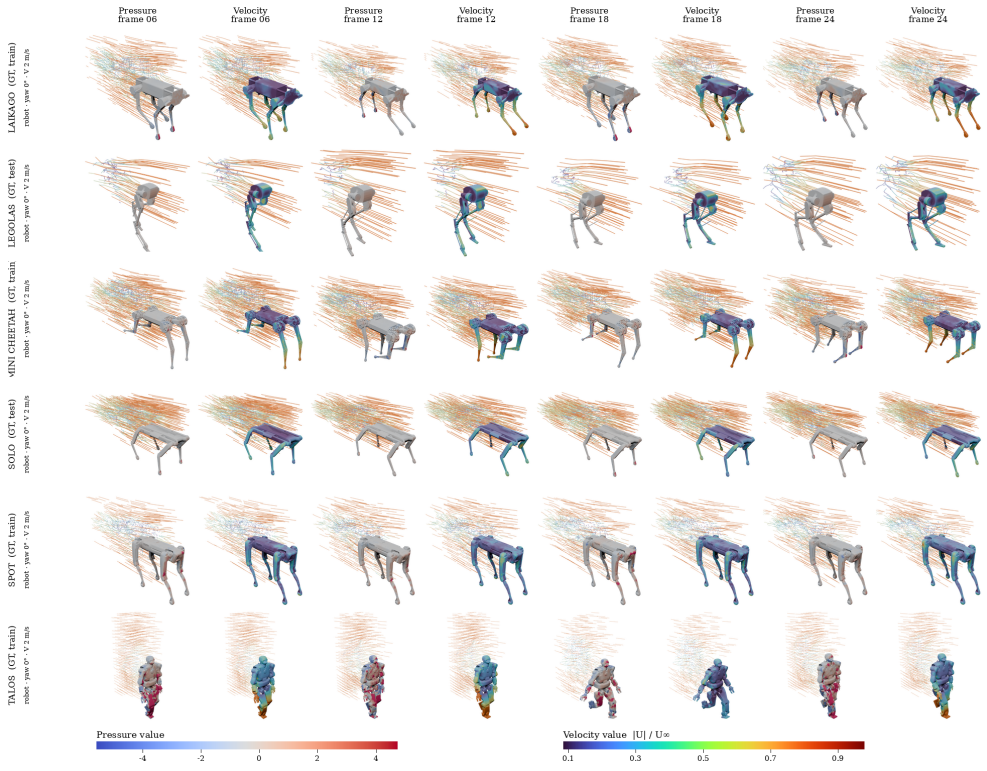


Figure 8: Ground-truth flow visualization from RoboCFD-4D. Six articulated robots are shown at rollout frames 6, 12, 18, and 24 under a 2 m/s inflow and $0°$ yaw. Each pair of columns shows pressure and normalized velocity magnitude $|\mathbf{U}|/U_\infty$ around the moving robot. Row labels indicate the robot and whether its morphology is included in training or held out for evaluation.

Quality control and dataset release. Quality control is applied at three complementary levels. Geometry and motion checks cover mesh validity, link-local SDF consistency, articulation, and swept-domain fit throughout the prescribed motion. Numerical checks cover temporal completeness, finite values in fluid cells, consistency between stored velocity vectors and speed magnitude,

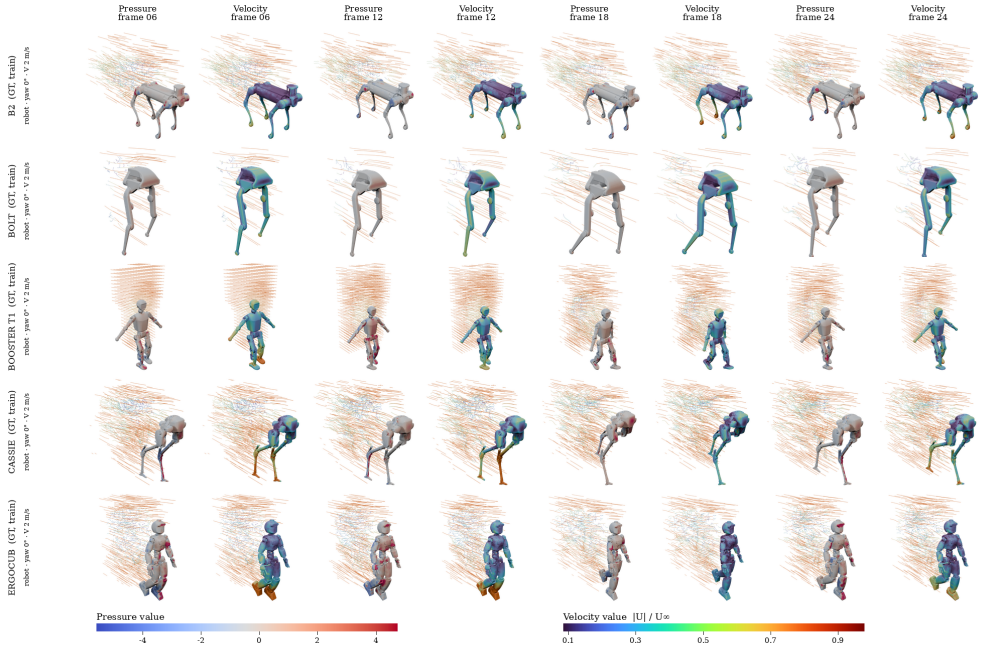


Figure 9: Ground-truth flow visualization from RoboCFD-4D. Five articulated robots are shown at rollout frames 6, 12, 18, and 24 under a 2 m/s inflow and 0° yaw. Each pair of columns shows pressure and normalized velocity magnitude $|\mathbf{U}|/U_\infty$ around the moving robot. Row labels indicate the robot and whether its morphology is included in training or held out for evaluation.

the prescribed pressure gauge, CFL ratios, normalized velocity divergence, and flux imbalance. Normalized velocity divergence and flux imbalance are subject to enforced rejection thresholds during dataset validation. Pressure-reference validation further requires a nonempty $\mathcal{P}_t$ for every output frame and a residual mean of the gauge-normalized pressure over $\mathcal{P}_t$ no greater than 10^{-4}.

Dataset-integrity checks verify surface-array dimensions, surface-normal lengths, pressure-unit consistency, and the finite-value policy. NaN values are permitted for Eulerian velocity and pressure inside solid cells and for near-surface samples whose interpolation stencils contain no positive valid-fluid weight; values expected to represent valid fluid observations must be finite. Publication requires an interpolation-valid fraction of at least 95% on average for each wind condition and at least 90% in every retained frame. Conditions that fail these gates are excluded until the underlying issue is corrected and the affected outputs are regenerated and revalidated.

A robot version is published only when all 16 configured wind conditions have successful execution records, complete 200-frame raw dense-field outputs, and the required 8192-point tracked near-surface data with passing quality checks. Partial simulations and incompatible pipeline versions are excluded. Benchmark sequences are constructed only after raw-run validation using the fixed zero-based temporal selection described above.

Versioned simulation directories retain source references, mesh and motion assets, simulation configurations, audit records, and artifact hashes alongside the generated HDF5 outputs. The HDF5 files contain the aligned volumetric and near-surface fields together with motion transforms and metadata, while the accompanying audit and quality-control records document the corresponding validation results.

C Additional Qualitative Results

We first visualize representative ground-truth trajectories from RoboCFD-4D in Fig. 8 and Fig. 9. These examples show how the articulated boundary and the surrounding velocity and pressure fields evolve over time across different robot morphologies. They provide a direct view of the prediction targets, independent of any learned model.

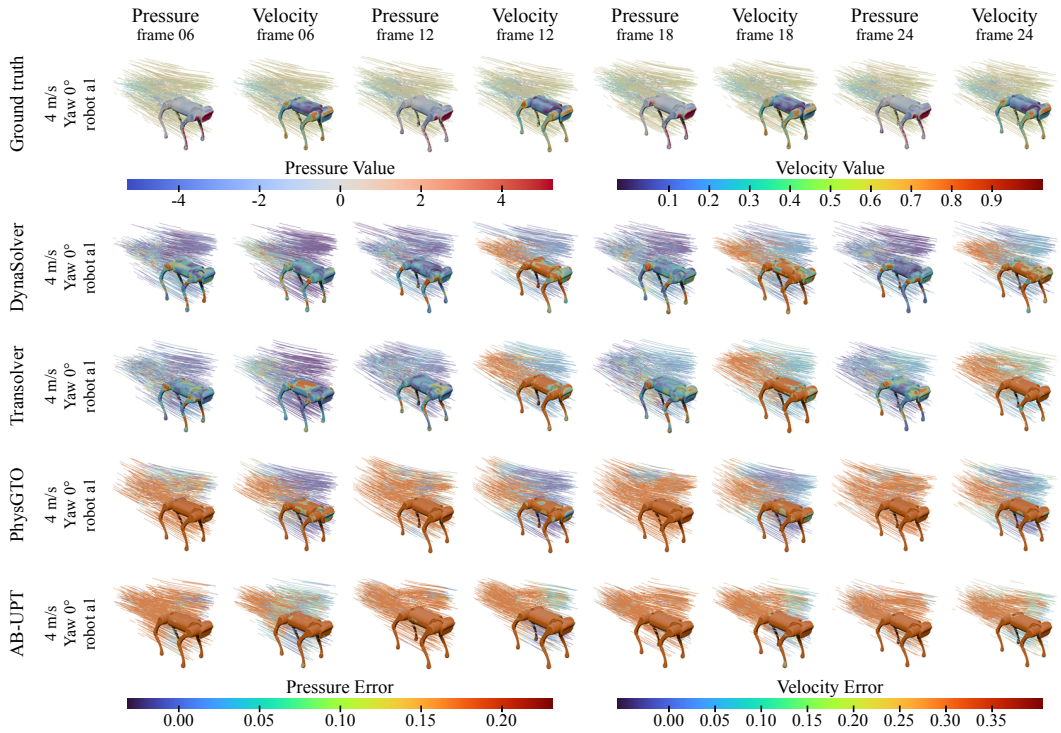


Figure 10: **Seen morphology under an unseen inflow condition.** Error maps of four models on a quadruped, evaluated at 4 m/s and 0° yaw. The morphology belongs to a family that the model sees during training, whereas this inflow condition is held out, so the figure isolates the effect of the flow regime with the boundary geometry in distribution. Columns alternate between the pressure and the velocity field at rollout steps 6, 12, 18 and 24, and all panels share one colour scale.

The qualitative figure in the main text reports a humanoid that is held out from training. This appendix adds two cases that isolate the two axes of generalization separately. Figure 10 keeps the morphology in distribution and changes the inflow: the quadruped belongs to a family that the model does see during training, but the case at 4 m/s and 0° yaw is held out, so a change in accuracy here is attributable to the flow regime rather than to an unfamiliar boundary. Figure 11 does the opposite: it holds out an entire humanoid body, which does not appear in training under any condition, and evaluates it within the range of inflow conditions seen during training, so a change in accuracy here is attributable to the geometry.

D Limitations and Future Work

Our results should be interpreted within the scope of the current benchmark and model design. While DynaSolver shows clear advantages in dynamic-boundary 4D CFD prediction, several limitations remain before the method can be considered broadly applicable.

Dataset scale and simulation scope. RoboCFD-4D is entirely simulation-derived and remains modest in both morphology diversity and operating-condition coverage. The reported accuracy therefore characterizes performance within this benchmark rather than real-world or industrial deployment. Extending the dataset to a broader range of embodiments, motions, and flow conditions will be necessary to assess robustness under more diverse settings.

Scalability to higher spatial resolutions. DynaSolver is designed to improve the efficiency of spatio-temporal CFD prediction, but it does not include dedicated mechanisms for very high-resolution throughput or memory scaling. Its efficiency is therefore established only at the spatial resolution evaluated in this work, and scaling to substantially denser CFD discretizations remains an open challenge.

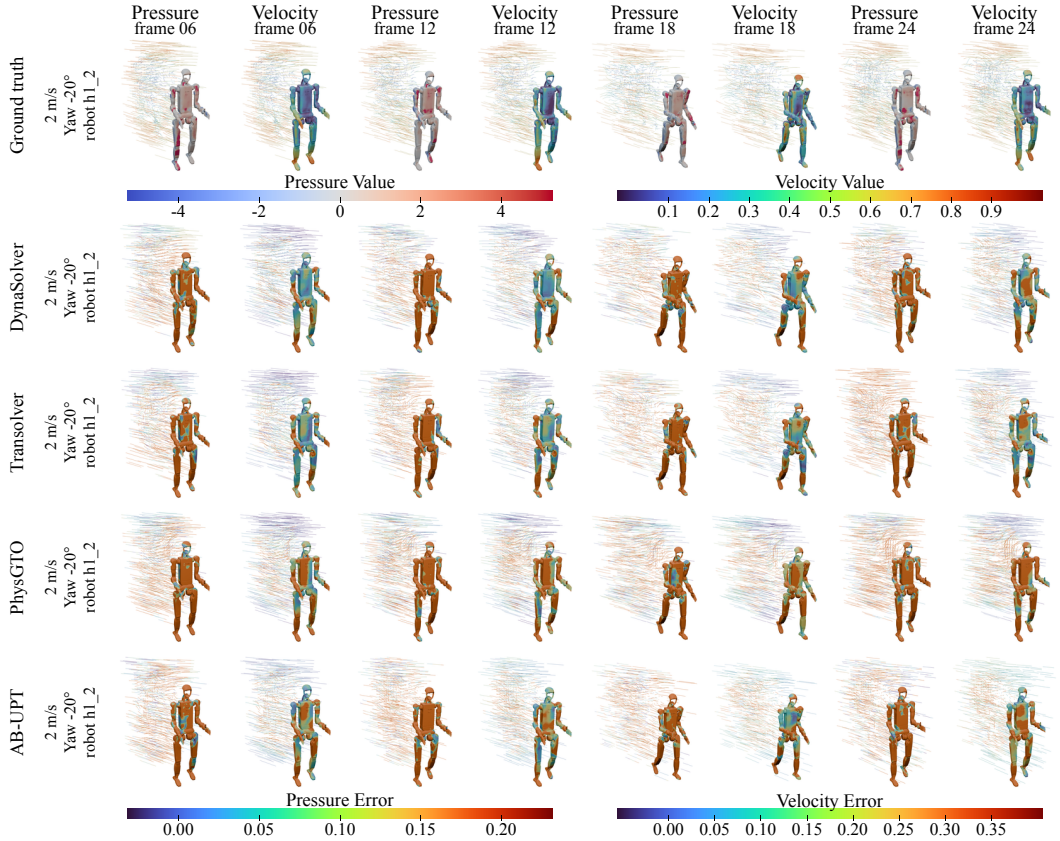


Figure 11: **Held-out morphology under an in-range inflow condition.** The same layout as Figure 10, for an additional humanoid body that is held out from training in all conditions, evaluated at 2 m/s and -20° yaw, which lies within the range of conditions the model was trained on. The inflow therefore remains in distribution and the figure isolates the effect of an unfamiliar morphology. Colour is shared by all panels of the figure.

Generalization to unseen geometries and more complex flow dynamics. Cross-morphology generalization remains challenging, with noticeably larger errors on robot geometries held out from training. A second failure mode emerges under stronger inflow conditions. As shown in Fig. 12, pressure errors increase markedly with inflow speed and become increasingly widespread throughout the wake at 6–8 $\mathrm{m/s}$. In these cases, DynaSolver does not fully capture the stronger temporal variability of the reference flow, leading to rapidly increasing rollout error. These observations indicate that the current model remains sensitive to shifts in both geometry and flow dynamics, particularly in strongly unsteady regimes.

Pressure prediction. Pressure remains substantially more challenging than velocity in long-horizon rollout. In particular, the non-learned persistence baseline is competitive with DynaSolver in average pressure error, indicating that the current model does not consistently improve pressure evolution over a no-evolution reference. Meanwhile, DynaSolver substantially improves velocity prediction and achieves the lowest pressure error among learned methods. Since the overall metric jointly measures three velocity channels and one pressure channel, its improvement is driven more strongly by the velocity components. Improving pressure dynamics remains an important direction for future work.

Motion realism. The current benchmark uses prescribed kinematic trajectories rather than dynamically generated locomotion. Most articulated motions are synthesized using smooth morphology-aware joint trajectories, and the root undergoes no net horizontal locomotion. This controlled setting isolates flow evolution induced by articulated boundary motion, but does not capture the full diver-

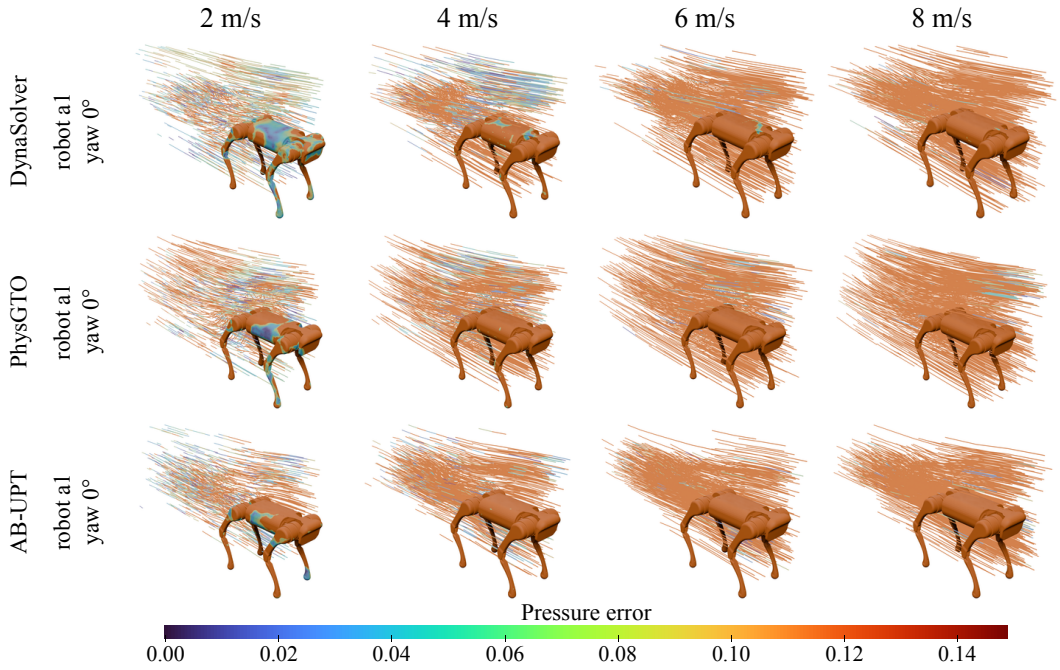


Figure 12: **Failure under stronger inflow conditions.** Absolute pressure errors $|p_{\text{pred}} - p_{\text{gt}}|$ at rollout step $t = 24$ for a quadruped at 0° yaw under inflow speeds of 2, 4, 6, and $8\,\mathrm{m/s}$. A shared color scale is used across all methods and conditions. While errors are relatively localized at low inflow speeds, they increase sharply and spread throughout the wake at higher speeds, indicating that strongly unsteady flow conditions remain challenging for current neural CFD surrogates.

sity of real robotic locomotion or two-way fluid–structure interaction. Extending RoboCFD-4D to measured or dynamically simulated motions is an important direction for future work.